INF1009 Assignment 1 Report

Abstract Simulation Engine

Team ID: P1-09

Name	Student ID	Email
Angel Zhang XinYu	2503541	2503541@sit.singaporetech.edu.sg
Pham Anh Bao Khang	2503012	2503012@sit.singaporetech.edu.sg
Timothy Chia Kai Lun	2501530	2501530@sit.singaporetech.edu.sg
Jaison Tan Yann Hann	2501161	2501161@sit.singaporetech.edu.sg

Course: INF1009 – Object Oriented Programming
Institution: Singapore Institute of Technology
Date of Submission: -

Table of Contents

1. Introduction....2
1.1 Overview of the Abstract Engine....3
1.2 Key Features Implemented....3
2. System Design and Implementation....4
2.1 Architectural Overview....4
2.1.1 Design Philosophy and Rationale....4
2.1.2 High-Level Architecture....4
2.1.3 Major Subsystems and Interactions....5
2.2 Managers and Their Responsibilities....6
2.2.1 EntityManager....6
2.2.2 SceneManager....7
2.2.3 MovementManager....7
2.2.4 CollisionManager....8
2.2.5 InputManager....9
2.2.6 RenderManager....10
2.3 Application of OOP Principles....11
2.3.1 Abstraction....11
2.3.2 Encapsulation....12
2.3.3 Inheritance....13
2.3.4 Polymorphism....13
2.3.5 Composition....14
2.3.6 Interfaces and Abstract Classes....14
2.4 Reflection on Engine Limitations....15
3. UML Diagram (Full Engine UML)....15
4. References....16
5. Team Contributions....17

1. Introduction

1.1 Overview of the Abstract Engine

The abstract simulation engine represents a modular, extensible framework designed to support applications through object-oriented principles. Built upon the libGDX framework, which provides rendering and input handling capabilities, the engine establishes a higher-level abstraction layer that enables the creation of diverse simulations and games without reimplementing core subsystems.

The engine's design utilizes Object Orientation Programming and SOLID principles. The engine's structure is organized into eight packages that map directly to the engine's main responsibilities. Core orchestration sits in `engine` package and includes the `Engine` class and manager hierarchy, state and lifecycle handling in the `scene package`, entity management in the `entity` package, collision detection and resolution in the `collision` package, movement and boundary constraints in the `movement` package, transformation capabilities in the `transform` package, input abstraction in the `input` package, and a queue-based rendering pipeline in the `render` package. This modular structure aligns with the Single Responsibility Principle by keeping each concern isolated and easier to extend independently.

The engine adopts an entity-capability model where objects or entities gain behaviors through interface implementation rather than through deep inheritance hierarchies. Entities requiring movement capabilities implement the `IMovable` interface, those participating in collision detection implement `ICollidable`, and those rendering to screen implement `IRenderable`. This capability-based design demonstrates the Interface Segregation Principle by ensuring that entities depend only on the specific interfaces they require and enables arbitrary combinations of capabilities without rigid class hierarchies.

1.2 Key Features Implemented

At a high level, the key features can be organized into architectural features (shown in Table 1) that establish the framework's structure and design patterns, and runtime features (shown in Table 2) that provide concrete functionality for simulations or games.

Feature	Description
Lifecycle hook management	Managers and scenes enforce consistent init/shutdown and enter/exit hooks through abstract base classes.
Separated entity access	Read and write operations on the entity store are handled through distinct interfaces, minimizing coupling.
Unified scene context	Scenes receive a single context object aggregating subsystem access, rather than direct manager references.
Logical action input	Simulation logic queries actions rather than platform-specific input codes, enabling runtime remapping.

Table 1. Architectural features

Feature	Description
Separated update/render pipeline	Update and render operations execute in distinct phases each frame.
Queue-based rendering	Drawable entities are submitted to a queue and processed in two passes each frame (update, then render).
Dimension-agnostic movement	Spatial transforms support arbitrary dimensions through float array representation.
Deferred scene transitions	Transition requests are queued and resolved at the start of the next frame to prevent mid-frame state mutations.

Table 2. Runtime features

2. System Design and Implementation

2.1 Architectural Overview

2.1.1 Design Philosophy and Rationale

The abstract engine adopts a class-based architectural approach organized around specialist manager classes. This design style was selected to maximize opportunities for demonstrating Object-Oriented principles, including inheritance hierarchies, polymorphism through method overriding, and encapsulation through abstract classes. The design deliberately prioritizes instructive clarity, clearly and simply illustrating inheritance relationships and polymorphic substitution to make these core concepts easier to grasp in an educational context.

2.1.2 High-Level Architecture

The system architecture is built around the `Engine` class, which serves as an orchestrator for composing collections of `IManager` and `IUpdatable` objects. A central structural component is the abstract `Manager` base class, which implements these interfaces to establish a uniform lifecycle contract across all specialist managers. The main engine loop sequentially triggers these managers to coordinate system flow. Crucially, because this is designed as a generic abstract simulation engine, these managers strictly handle orchestration and lifecycle routing rather than encapsulating behavioral logic. This architectural separation decouples the engine's core from domain-specific operations, allowing a variety of simulations. Subsystems primarily interact by querying the unified `SceneContext` and the central `EntityManager`, ensuring that managers remain decoupled from one another while still operating on the same shared entity state.

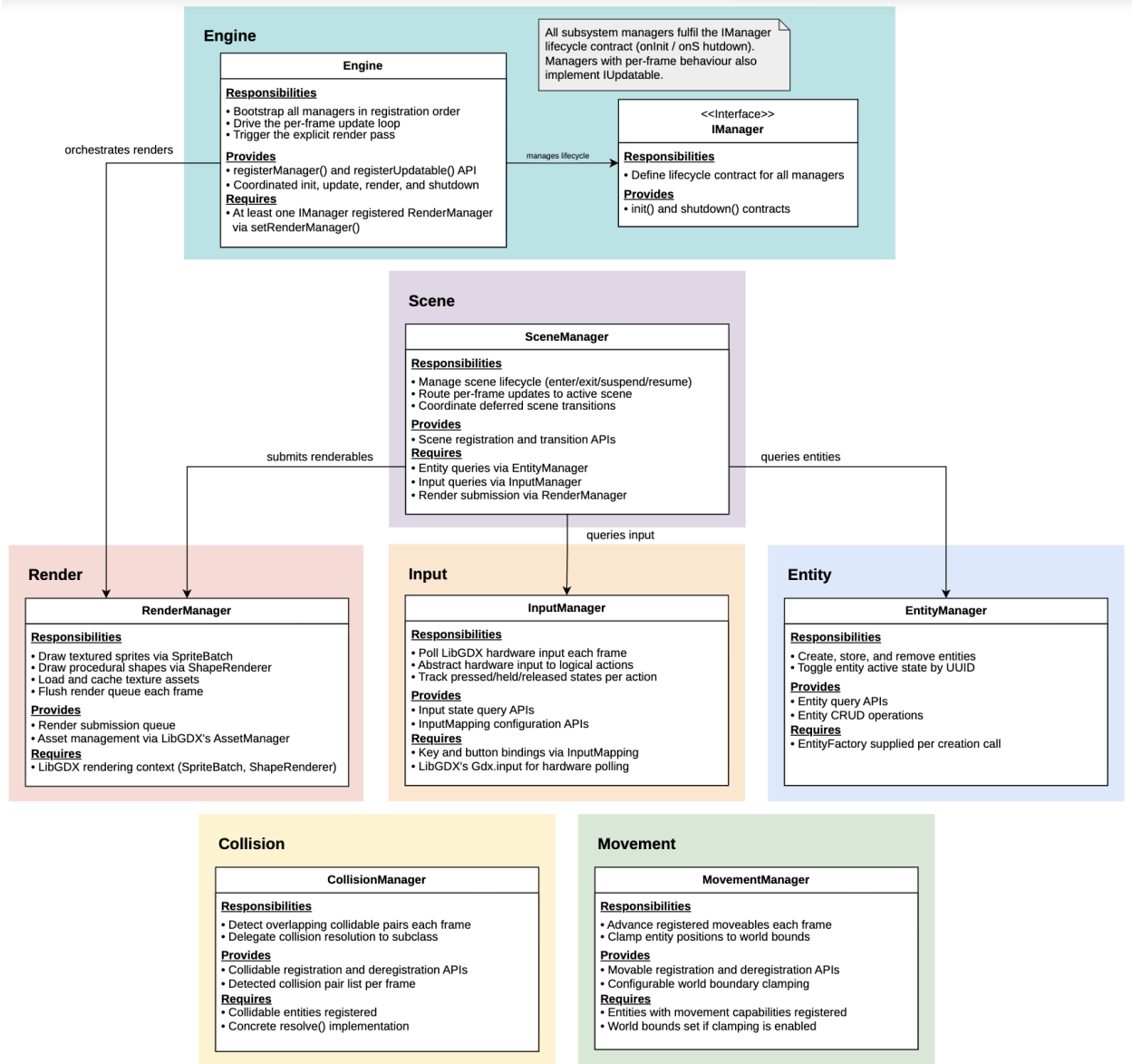


Figure 1. *Abstract Engine High-Level Architecture*

The figure illustrates the six manager subsystems and their dependencies. `SceneManager` coordinates access to Entity, Render, and Input systems, while Movement and Collision managers handle physics interactions.

2.1.3 Major Subsystems and Interactions

The engine's operations can be understood through four primary interaction patterns: initialisation, per-frame updates, rendering, and scene transitions.

Initialisation: The application creates an `Engine` instance and registers managers through `registerManager()`. Registration order is significant as managers with dependencies must be registered after their own dependencies. Once all managers are registered, `Engine.init()` iterates the manager collection and invokes `init()` on each in registration order.

Per-frame update: Each frame, `Engine.update(deltaTime)` iterates registered `IUpdatable` managers in registration order, invoking each manager's `update()` method.

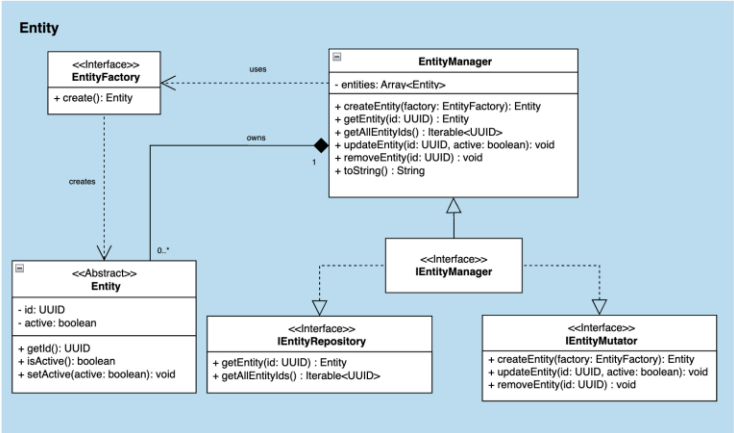
Rendering: The application calls `Engine.render()`, which delegates to the `RenderManager`. The render manager processes entities submitted to its queue during the previous frame's `submitRenderable()` callbacks, in two passes: textured items via `SpriteBatch`, then procedural items via the `renderCustomProcedural` hook.

Scene transitions: When a scene calls `context.changeScene(key)`, the request is stored rather than executed immediately. At the beginning of the next frame's update, the scene manager calls `onExit()` on the current scene, looks up the new scene, and calls its `onEnter()` callback, preventing mid-frame state changes.

2.2 Managers and Their Responsibilities

The abstract engine delegates specific concerns to six specialized manager classes, each responsible for a distinct aspect of simulation functionality. This section describes each manager's purpose, the objects it manages, its key methods, and the design patterns it demonstrates.

2.2.1 EntityManager



The `EntityManager` serves as the central registry for all simulation objects. It implements a unified contract `IEntityManager` that extends `IEntityRepository` for read-only queries and `IEntityMutator` for write operations. Its key responsibilities are:

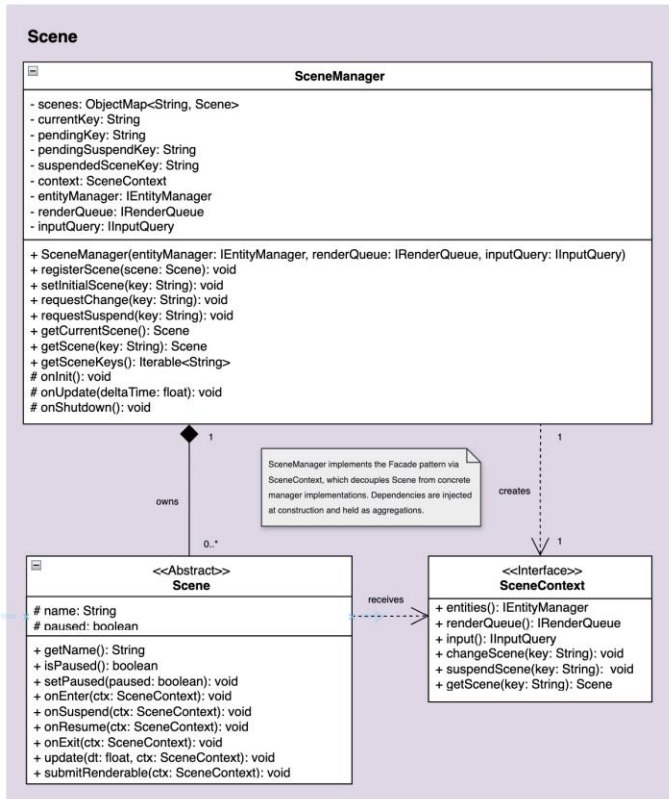
- Maintain a collection of all `Entity` instances with unique `UUID` identifiers
- Provide entity queries through `getEntity(UUID)`
- Create entities via `createEntity(EntityFactory)`
- Remove entities through `removeEntity(UUID)`

Primary methods include: `createEntity()`, `getEntity()`, `updateEntity()`, `removeEntity()`, and `getAllEntityIds()`.

The factory-based creation method decouples entity creation from storage. Rather than directly instantiating entities, the manager accepts an `EntityFactory` functional interface that defines a `create()` method, enabling clients to create any Entity subclass without the manager requiring knowledge of those subclasses.

Its dual-interface design means the manager can be passed to read-only clients through `IEntityRepository` or to lifecycle clients through `IEntityMutator`, preventing scenes from accidentally mutating the entity store through the query interface.

2.2.2 SceneManager



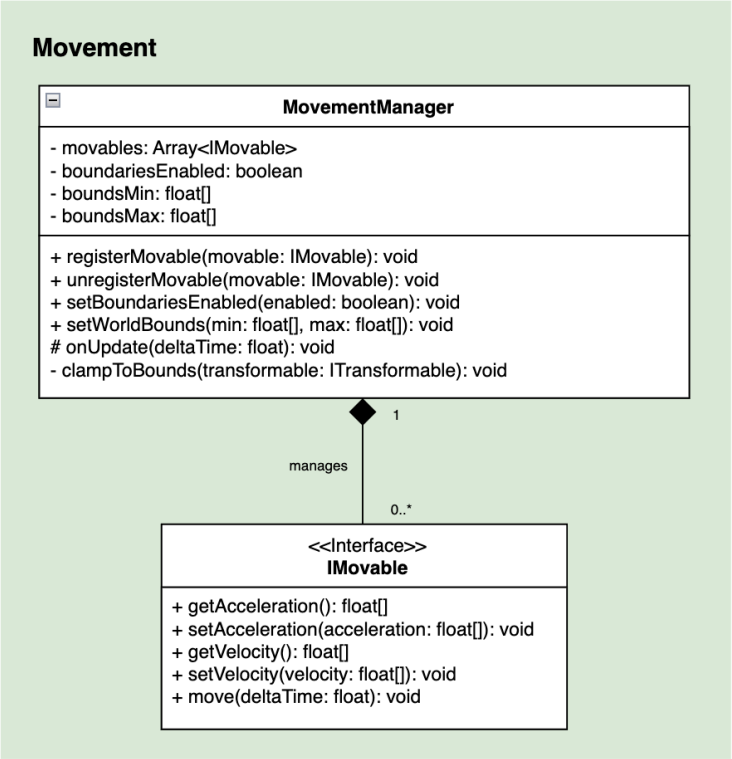
The `SceneManager` coordinates transitions between distinct simulation states, each represented by a `Scene` subclass. It extends `UpdatableManager`, participating in per-frame updates to drive scene lifecycle callbacks and resolve pending transitions. Its key responsibilities are:

- Maintain a collection of named scenes and the currently active scene
- Handle deferred scene transitions through `changeScene()` and `suspendScene()`
- Drive scene lifecycle through `onEnter()`, `onExit()`, `onSuspend()`, `onResume()` callbacks
- Aggregate dependencies that the `SceneContext` interface provides

Primary methods include: `registerScene()`, `changeScene()`, `suspendScene()`, `getScene()`, and `onUpdate()`.

Support for suspend and resume semantics enables user interface-based scenes like pause menus to overlay a previous scene without full scene reconstruction. When a scene is suspended, its update loop pauses, but rendering continues.

2.2.3 MovementManager



The MovementManager updates spatial positions for entities implementing IMovable and enforces boundary constraints. It extends UpdatableManager and demonstrates a dimension-agnostic design. Its key responsibilities are:

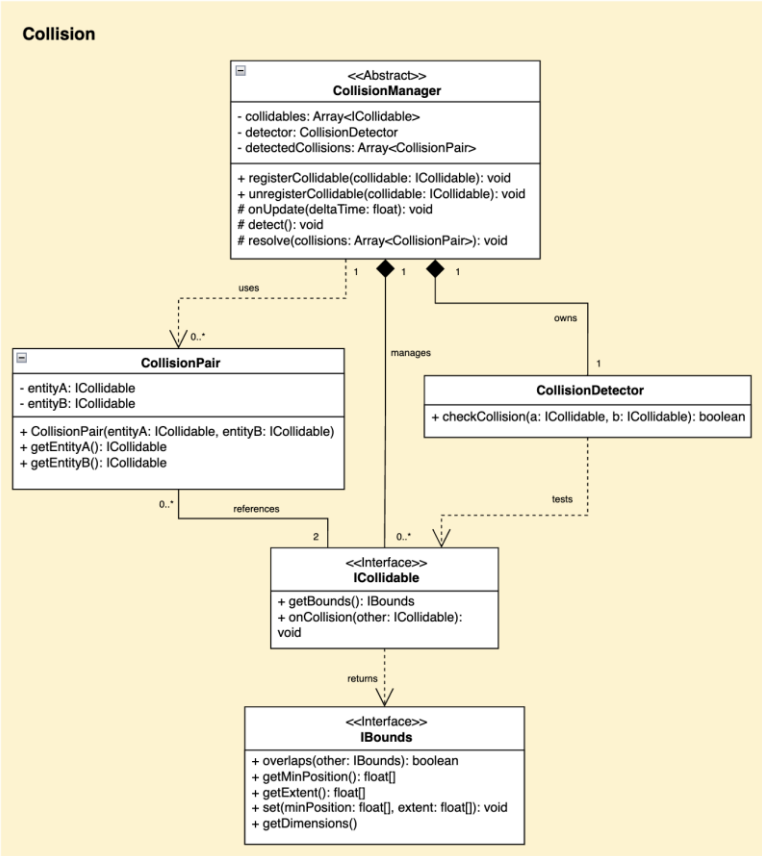
- Updating positions for all registered IMovable entities via their `move(deltaTime)` methods
- Enforce world boundaries through configurable bounds with setWorldBounds()
- Perform validation on boundary configuration

Primary methods include: `registerMovable()`, `unregisterMovable()`, `setWorldBounds()`, `setBoundariesEnabled()`, and `onUpdate()`.

The dimension-agnostic design supports arbitrary spatial dimensions as positions and sizes are represented as generic float arrays, enabling support for 2D, 3D, or higher-dimensional spaces.

The boundary validation in `setWorldBounds()` demonstrates defensive programming, checking that arrays are non-null, have matching lengths, and satisfy `min[i] <= max[i]` for each dimension, catching configuration errors during initialization rather than at runtime.

2.2.4 CollisionManager

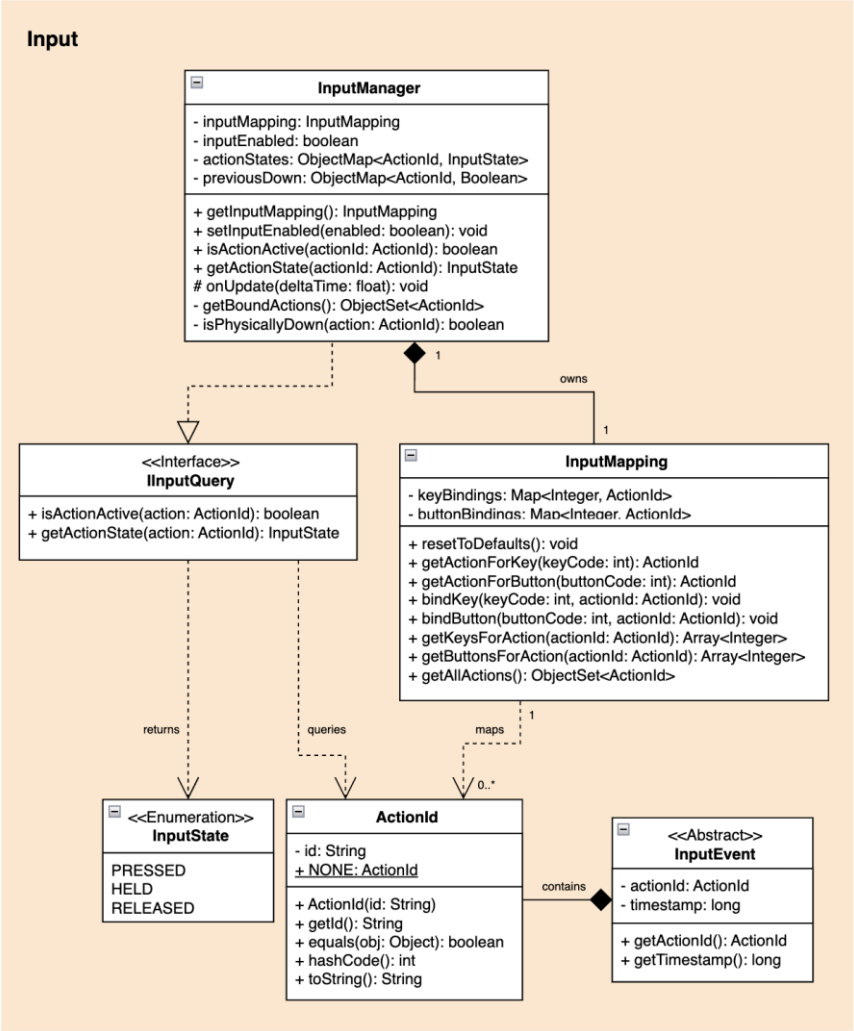


The CollisionManager performs generic collision detection between ICollidable entities and delegates resolution to an abstract method, demonstrating the Open-Closed Principle. Its key responsibilities are:

- Broad-phased detection comparing all collidable pairs
- Narrow-phased AABB intersection tests via stateless `CollisionDetector` utility class
- Abstract `resolve(Array<CollisionPair>)` method for extensible resolution strategies

Primary methods include: `registerCollidable()`, `unregisterCollidable()`, `detect()`, `resolve()`, and `onUpdate()`. The separation of detection from resolution enables different collision resolution strategies without modifying detection code. The abstract `resolve()` method allows subclasses to implement callback-based resolution.

2.2.5 InputManager



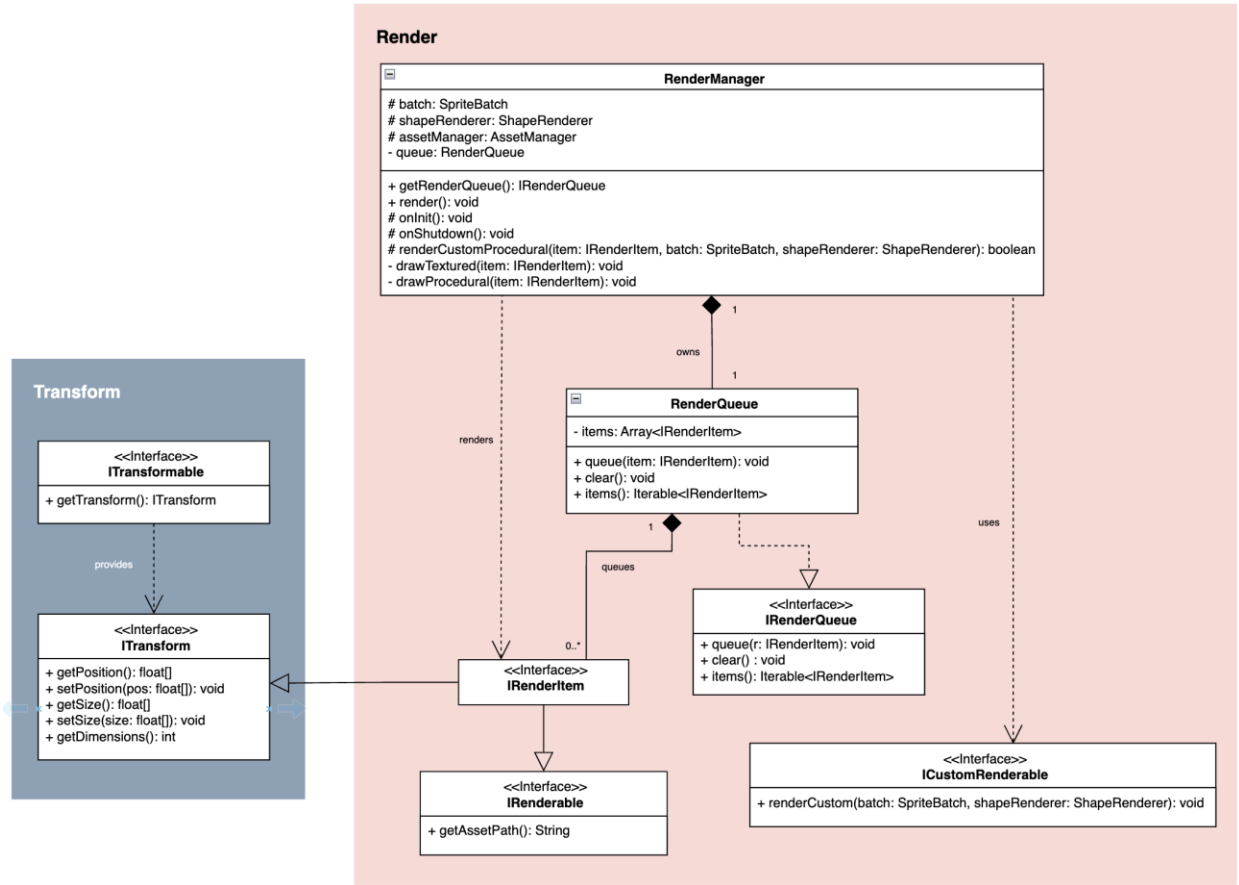
The `InputManager` abstracts platform-specific input mechanisms behind a logical action-based interface. It extends `UpdatableManager` and implements `IInputQuery`. Its key responsibilities are:

- Maintain an `InputMapping` object associating action identifiers with platform-specific input codes
- Track state transitions such as pressed, held, or released by comparing current and previous frame states
- Provide action queries through `isActionActive()` and `getActionState()`

Primary methods include: `mapAction()`, `isActionActive()`, `getActionState()`, and `onUpdate()`.

The action-based abstraction enables downstream game code to query logical actions such as moving left/right without depending on specific keys or controller buttons. The frame-based polling approach ensures input state is sampled consistently at the beginning of each frame, preventing race conditions.

2.2.6 RenderManager



The `RenderManager` provides a rendering pipeline that processes a queue of `IRenderItem` instances. It manages libGDX rendering resources including `SpriteBatch`, `ShapeRenderer`, and `AssetManager`. Its key responsibilities are:

- Maintain a `RenderQueue` that accumulates drawable items submitted by scenes
- Process the queue in two passes: textured items via `SpriteBatch`, then procedural items via `renderCustomProcedural()`
- Manage rendering resource lifecycle through `onInit()` and `onShutdown()`

Primary methods include: `render()`, `renderCustomProcedural()`, `onInit()`, and `onShutdown()`.

The queue-based architecture decouples scene logic from rendering concerns. Scenes submit items without knowledge of batching, texture management, or draw call optimizations. The `renderCustomProcedural()` method enables extension for text rendering or UI rendering without modifying the core pipeline.

2.3 Application of OOP Principles

2.3.1 Abstraction

Abstractions conceal implementation details behind simplified interfaces, exposing only essential features. The engine demonstrates abstraction through abstract classes that partially implement types that cannot be instantiated directly and interfaces that define contracts without specifying implementation details.

At the engine level, `IManager` and `IUpdatable` define the two core behavioral contracts which are for lifecycle management and per-frame updating that all managers must satisfy. The Manager abstract class

implements these using the Template Method pattern, delegating to `onInit()` and `onShutdown()` hooks, while `UpdatableManager` extends this by declaring an abstract `onUpdate()` method, forcing subclasses to supply their own per-frame logic without exposing the surrounding lifecycle operations.

Within entity management, the Entity abstract class conceals the mechanics of identity and activation state, and the functional interface `EntityFactory` abstracts entity construction entirely, allowing `EntityManager` to create any concrete entity subclass without depending on it. `EntityManager` is further split into `IEntityRepository` and `IEntityMutator`, abstracting the two concerns independently. The input system similarly uses `IInputQuery` to hide physical device polling behind a logical action model, and `InputEvent` is declared abstract to capture shared event properties whilst deferring concrete event types to the application layer.

Spatial concerns are abstracted through `ITransform` and `ITransformable`, which represent positional state using dimension agnostic float arrays, making the engine portable across arbitrary dimensional contexts. This extends to the collision management subsystem, where `IBounds` abstract any bounding volume shape behind an `overlaps()` contract, and `ICollidable` abstracts both how an entity exposes its bounds and how it responds to a collision. `CollisionManager` is declared abstract to separate the stable detection phase from the variable resolution strategy, which subclasses must implement via the abstract `resolve()` method.

2.3.2 Encapsulation

Encapsulation restricts access to internal state, exposing it only through controlled interfaces. The engine applies this consistently through access modifiers, final method contracts, and package-level visibility.

Throughout all specialist manager subsystems, internal collections such as the entity list in `EntityManager`, the collidable registry in `CollisionManager`, the moveable registry in `MovementManager`, and the scene registry in `SceneManager`, are declared private final. This prevents external code from iterating or mutating them directly, funneling all access through dedicated registration and query methods. Similarly, transition state attributes in `SceneManager` such as current, pending, and suspended scene keys are entirely private, with callers only able to trigger transitions through `requestChange()` and `requestSuspend()`.

Fields representing configuration and runtime flags follow the same pattern. Engine exposes its `RenderManager` reference only through a guarded setter that rejects null arguments. `MovementManager` validates world boundary arrays before assignment in `setWorldBounds()`, enforcement that would be bypassed by direct field access. Manager holds an initialized flag that is exclusively managed by its final `init()` and `shutdown()` methods, preventing subclasses from corrupting lifecycle state. `UpdatableManager` similarly declares `update()` as final, locking the public contract and forcing subclasses to implement the protected `onUpdate()` hook rather than overriding the dispatch path directly.

Value objects across the engine like `CollisionPair`, `ActionId`, and `InputEvent`, declare all fields as private final, making them immutable after construction and exposing data only through read-only getters. Entity further restricts its constructor to protected, and `InputEvent` does the same, preventing direct instantiation from outside the class hierarchy and funneling creation through designated mechanisms.

Internal algorithms are hidden behind private methods throughout. `InputManager` keeps hardware polling in `isPhysicallyDown()` and action discovery in `getBoundActions()` are entirely private. `MovementManager` encapsulates its boundary clamping in a private `clampToBounds()` method. `RenderManager` splits its rendering logic across private `drawTextured()` and `drawProcedural()` helpers, exposing only the single public `render()` method. `SceneManager` wraps suspend/resume book-keeping in three private helpers invisible to callers.

At the package level, `RenderQueue` is declared without an access modifier, making it package private. Code outside the render package cannot reference the concrete type at all, and only ever interacts with it through the `IRenderQueue` interface.

2.3.3 Inheritance

Inheritance establishes an "IS-A" relationship where subclasses extend superclasses, inheriting state and behaviour. The engine's manager hierarchy is one clear application of inheritance.

Manager serves as the base abstract class, providing shared lifecycle state and the final methods init() and `shutdown()` to all subclasses. `UpdatableManager` extends Manager to introduce the update loop contract, inheriting the lifecycle infrastructure while adding the abstract onUpdate() method, establishing a second tier from which remaining specialist managers (`InputManager, MovementManager, SceneManager`, and `CollisionManager`) all extend `UpdatableManager`, inheriting both lifecycle and update loop behaviour without reimplementing either. This two-level class hierarchy eliminates duplication across six manager subclasses while preserving a clean separation between managers that update and those that do not.
At the interface level, `IRenderItem` inherits from both `IRenderable` and `ITransformable`, composing two independent contracts into a single type that the render queue can work with uniformly. This means any type satisfying `IRenderItem` is guaranteed to expose both its visual representation and its spatial state without requiring callers to cast or query each concern separately.

The engine also defines three abstract classes: `Entity`, `Scene`, and `InputEvent`, which carry no parent themselves but are designed explicitly for inheritance by the application layer. Each encapsulates shared state and enforces a protected constructor, ensuring subclasses are the only permitted instantiation path while leaving concrete behavior to be defined by the inheriting type.

2.3.4 Polymorphism

Polymorphism enables objects of different types to be treated uniformly through a common interface. The engine demonstrates polymorphism extensively through method overriding and interface implementation. Method overloading is deliberately left to the application layer, where concrete subclasses can introduce overloaded variants suited to their domain.

Again, the manager hierarchy is the main application of polymorphism as every specialist manager that inherits from `UpdatableManager` overrides the `onUpdate()` hook, substituting distinct per-frame behaviour behind the same method signature. The Engine class exploits this by storing all managers as `IManager` and all updatables as `IUpdatable`, dispatching lifecycle and update calls uniformly through interface references with no knowledge of the concrete types involved.

Across the specialist manager's subclasses, collections are consistently typed to interfaces rather than concrete classes. `CollisionManager` stores `Collidable` references and dispatches both the bounds check and the collision callback entirely through the interface contract. `MovementManager` stores `IMovable` references and dispatches `move()` per element, narrowing to `ITransformable` at runtime where boundary clamping applies. `RenderManager` iterates the queue as `IRenderItem` references, and further narrows to `ICustomRenderable` to dispatch custom rendering where the item satisfies the interface. `SceneManager` holds scenes as the abstract Scene type and dispatches `update()` and `submitRenderable()` to whichever concrete scene is active, while its own dependencies are stored `as IEntityManager, IRenderQueue`, and `IInputQuery`, decoupling it entirely from the concrete implementations behind each.

Abstract classes also act as override points beyond simple method dispatch. `CollisionManager` declares `resolve()` as abstract, making the resolution strategy itself polymorphically determined by the concrete subclass. The non-abstract `detect()` method and `RenderManager's renderCustomProcedural()`

function as Template Method hooks, providing default behavior that subclasses may replace without altering the surrounding algorithm.

2.3.5 Composition

Composition models a "HAS-A" relationship whereby objects contain references to other objects as fields, delegating responsibilities rather than inheriting them. The engine applies this throughout, both by constructing and owning objects internally and by accepting them as injected dependencies.

At the engine level, Engine composes two owned collections: Array<IManager> and Array<IUpdatable>, through which it drives the full lifecycle and update loop without any knowledge of the concrete types involved. The RenderManager reference is held as a separate field and injected via a guarded setter, decoupling the engine from how rendering is constructed externally.

Each specialist manager composes the objects it needs to carry out its responsibility. CollisionManager owns a CollisionDetector instance created internally, delegating the overlap check to it, and maintains a separate Array<CollisionPair> as a per-frame temporary buffer. InputManager owns an InputMapping object that encapsulates all key and button bindings, alongside two ObjectMap fields that track current and previous action states for state-transition derivation. RenderManager creates and fully lifecycle-manages three libGDX resources: SpriteBatch, ShapeRenderer, and AssetManager, alongside an owned RenderQueue instance that it exposes externally only through the IRenderQueue interface.

SceneManager demonstrates composition through dependency injection. Its constructor accepts IEntityManager, IRenderQueue, and IInputQuery references, all stored as final interface-typed fields. These are then closed over by an anonymous SceneContext implementation assembled in onInit(), which presents a unified context object to active scenes without exposing the underlying managers directly.

Value objects in the engine also showcase composition. CollisionPair is composed entirely of two final ICollidable references injected via its constructor, making its identity wholly derived from the objects it bundles. InputEvent composes an ActionId field in the same manner. Entity composes a UUID generated at construction time to delegate identity entirely to that object, with ActionId following the same pattern for its backing String.

2.3.6 Interfaces and Abstract Classes

The Interface Segregation Principle states that no class should be forced to depend on methods it does not use. The engine upholds this by keeping each interface narrow, purpose-specific, and independently implementable, so that a class only takes on the obligations it genuinely requires.

The clearest application is the separation of IManager and IUpdatable. Managers that do not participate in the update loop, such as RenderManager and EntityManager. satisfy only IManager and carry no obligation to implement update(). EntityManager in particular has no per-frame logic of its own; entity state is updated reactively in response to events within a scene, rather than on a fixed tick. The two concerns are only combined where a class genuinely needs both, through UpdatableManager. Because the engine operates entirely against these interfaces, any object satisfying IManager or IUpdatable can be registered and driven by the engine without it needing to be a subclass of any existing manager, making the engine straightforwardly extensible to new subsystems beyond current implementations.

The same principle governs entity access: `EntityManager` is split into `IEntityRepository` and `IEntityMutator`, separating read and write concerns into independent contracts. These are composed by `IEntityManager`, which extends both and serves as the dependency for consumers that require full access,

such as `SceneContext`. Consumers that genuinely need only one side of the contract can depend on the narrower interface directly, taking on no obligation to implement or accept the other.

The render subsystem applies the same pattern across three distinct capability interfaces. `IRenderable` is a minimal contract with a single method, `getAssetPath()`, which determines whether an entity is drawn via texture or procedurally. `ITransformable` is defined separately in the transform package and concerns only spatial state. Neither interface implies the other; `IRenderItem` composes them for the render queue specifically, and only entities entering that queue need to satisfy both. `ICustomRenderable` is an entirely opt-in third interface for entities that own their own rendering logic entirely, with no obligation to implement it otherwise. This means implementing classes acquire only the rendering obligations relevant to them, rather than satisfying a single monolithic renderable contract.

`IMovable` follows the same pattern in the movement subsystem. It defines acceleration, velocity, and a single `move()` method, with no spatial or rendering obligations attached. An entity participates in movement solely by implementing this interface, and the use of generic float arrays means the contract is not tied to any fixed coordinate space. `SceneContext` serves as a deliberately restricted view of the engine presented to active scenes. Rather than passing scenes with a reference to `SceneManager` or any concrete manager, the context exposes only `entities()`, `renderQueue()`, `input()`, `changeScene()`, `suspendScene()`, and `getScene()`. Scenes receive precisely the capabilities they need to operate within a frame, with no access to internal manager state, registration methods, or lifecycle controls.

2.4 Reflection on Engine Limitations

Although this engine's design provides a clean separation of concerns across its subsystems, several structural limitations emerge from its current design.

The Engine class initialises managers in registration order and shuts them down in reverse but provides no mechanism for declaring or resolving dependencies between managers. If one manager depends on another being fully initialised before it. For instance, `SceneManager` depending on `EntityManager`, the caller must manually register them in the correct sequence. No enforcement or validation exists to detect an incorrect ordering at runtime, making the engine fragile under a non-trivial registration sequence.

The default collision detection algorithm in `CollisionManager.detect()` iterates all unique pairs of registered collidables, giving it $O(n^2)$ complexity. While the method is explicitly designed to be overridden with a more efficient implementation, no spatial partitioning is provided out of the box. In scenes with many collidable entities, this becomes a practical performance concern.

`IRenderItem` carries no depth or layer field. Draw order is determined entirely by the order in which items are submitted to the render queue inside each scene's `submitRenderable()` method. There is no engine-level z-ordering mechanism, so maintaining a consistent and correct draw order across a complex scene is left entirely to the application layer.

Finally, `RenderManager` is tightly coupled to libGDX, composing `SpriteBatch`, `ShapeRenderer`, and `AssetManager` directly as typed fields. The rendering subsystem cannot be replaced or ported to an alternative backend without modifying RenderManager itself, limiting the engine's portability beyond libGDX-based targets.

3. UML Diagram (Full Engine UML)

Include complete UML covering:

- Entities / Components
- Managers
- Scene / Core System
- Relationships (IS-A, HAS-A)
- Dependencies
- Interfaces / abstract classes

Full UML Link:
https://drive.google.com/file/d/1rlkXf82KvaBHJz-I3E3KrvDktaTPPtQ-/view?usp=drive_link

Full UML PDF:

INF1009-P1-09-UML.pdf

4. References

1. UML class diagram tutorial. (n.d.). https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/

5. Team Contributions

Name	Contributions
Angel Zhang XinYu	CollisionManager, Demo Code, UML Diagram, Report
Pham Anh Bao Khang	InputManager, MovementManager, UML Diagram, Report
Timothy Chia Kai Lun	EntityManager, Engine, UML Diagram, Report
Jaison Tan Yann Hann	SceneManager, RenderManager, UML Diagram, Report
AI	Architectural suggestions, clarification support, ideation for unique feature